

Lab 2: Deadlock Avoidance Using Banker's Algorithm

Theory

A deadlock is a situation in an operating system where a group of processes becomes permanently blocked because each process is waiting for a resource that is held by another process in the group. As a result, none of the processes can continue execution, causing the system to halt indefinitely unless corrective measures are taken.

Necessary Conditions for Deadlock

A deadlock can occur only if the following four conditions hold simultaneously:

- **Mutual Exclusion:** At least one resource is non-shareable and can be used by only one process at a time.
- **Hold and Wait:** A process holding one or more resources requests additional resources that are currently allocated to other processes.
- **No Preemption:** Resources cannot be forcibly taken away from a process; they must be released voluntarily.
- **Circular Wait:** A circular chain of processes exists where each process is waiting for a resource held by the next process in the chain.

Methods of Handling Deadlocks

Operating systems use one or more of the following strategies:

- **Deadlock Prevention:** Ensures that at least one of the necessary conditions for deadlock cannot occur.
- **Deadlock Avoidance:** Allocates resources only if doing so keeps the system in a safe state.
- **Deadlock Detection:** Allows deadlocks to occur and periodically checks for their presence.
- **Deadlock Recovery:** Terminates processes or preempts resources to recover from a detected deadlock.

Banker's Algorithm

The Banker's Algorithm, proposed by Edsger W. Dijkstra, is a deadlock avoidance algorithm used to determine whether granting a resource request will keep the system in a safe state. The algorithm checks the availability of resources before allocation to ensure that all processes can complete without causing a deadlock.

Data Structures Used

The algorithm maintains the following data structures:

- **Allocation Matrix:** Number of resources of each type currently allocated to each process.
- **Maximum Matrix:** Maximum number of resources of each type that each process may require.
- **Available Vector:** Number of available instances of each resource type.
- **Need Matrix:** Remaining resources required by each process, calculated as:

$$\text{Need} = \text{Maximum} - \text{Allocation}$$

Working of the Algorithm

The Banker's Algorithm performs the following steps:

1. Calculate the Need Matrix for all processes.
2. Compare the Need of each unfinished process with the Available resources.
3. If a process's needs can be satisfied, assume it completes execution and releases its allocated resources.
4. Update the Available resources and repeat the process for the remaining processes.
5. If all processes can finish successfully, the system is in a safe state and a safe sequence exists.
6. If no process can proceed at any stage, the system is in an unsafe state, and granting the resource request may eventually lead to a deadlock.

Advantages

- Prevents deadlocks before they occur.
- Ensures safe allocation of resources.
- Determines whether a safe execution sequence exists.
- Suitable for systems where the maximum resource requirement of each process is known in advance.

Limitation

- Requires prior knowledge of the maximum resource requirements of every process.
- May reduce resource utilization because some requests are denied to maintain system safety.
- Becomes computationally expensive as the number of processes and resources increases.

Source Code:

```
allocation = [  
    [0, 1, 0],  
    [2, 0, 0],  
    [3, 0, 2],  
    [2, 1, 1],  
    [0, 0, 2],  
]
```

```
maximum = [  
    [7, 5, 3],  
    [3, 2, 2],  
    [9, 0, 2],  
    [2, 2, 2],  
    [4, 3, 3],  
]
```

```
available = [3, 3, 2]
```

```
def bankers_algorithm(allocation, maximum, available):  
    process_count = len(allocation)  
    resource_count = len(available)  
  
    need = [  
        [maximum[i][j] - allocation[i][j] for j in range(resource_count)]  
        for i in range(process_count)  
    ]  
  
    work = available.copy()  
    finished = [False] * process_count  
    safe_sequence = []
```

```
    while len(safe_sequence) < process_count:
```

```
found_process = False
```

```
for i in range(process_count):
```

```
    if not finished[i] and all(  
        need[i][j] <= work[j] for j in range(resource_count)  
    ):
```

```
        for j in range(resource_count):  
            work[j] += allocation[i][j]
```

```
        finished[i] = True
```

```
        safe_sequence.append(f"P {i}")
```

```
        found_process = True
```

```
if not found_process:
```

```
    return False, []
```

```
return True, safe_sequence
```

```
is_safe, sequence = bankers_algorithm(allocation, maximum, available)
```

```
if is_safe:
```

```
    print("System is in a safe state.")
```

```
    print("Safe sequence:", " -> ".join(sequence))
```

```
else:
```

```
    print("System is not in a safe state.")
```

Output

Process	Allocation (A, B, C)	Maximum (A, B, C)	Available (A, B, C)
P0	[0, 1, 0]	[7, 5, 3]	[3, 3, 2]
P1	[2, 0, 0]	[3, 2, 2]	
P2	[3, 0, 2]	[9, 0, 2]	
P3	[2, 1, 1]	[2, 2, 2]	
P4	[0, 0, 2]	[4, 3, 3]	

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS
System is in a safe state.
Safe sequence: P1 -> P3 -> P4 -> P0 -> P2
○ (base) PS D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works>
```

When available = [0, 0, 0]:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS
System is not in a safe state.
○ (base) PS D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works>
```

Conclusion

In this lab, Banker's Algorithm was implemented to avoid deadlock by checking whether the system remains in a safe state before allocating resources. Using the allocation, maximum demand, and available resource values, the program calculated a safe sequence of processes. This demonstrates how proper resource allocation can prevent deadlocks and ensure that all processes complete successfully.

Lab 3: Windows Batch Script Automation for Local and Remote Repository Synchronization

Theory

Windows batch scripting is a method of automating repetitive command-line tasks in Windows through files with the .bat extension. File and folder synchronization maintains a duplicate copy of data from a source location to a destination location. The XCOPY command supports this process by copying files and directories, including subdirectories when appropriate options are used. Automating synchronization helps reduce manual effort and supports regular data backup or replication.

Windows Task Scheduler is a system utility that runs a program or script automatically at a defined time or interval. When a batch script is scheduled, the synchronization process can occur periodically without requiring user interaction. This improves reliability by ensuring that changes in the source directory are copied to the destination directory regularly.

Git is a distributed version-control system used to track changes in files and synchronize work between local and remote repositories. The basic synchronization workflow consists of modifying files locally, staging and committing the changes, and using git push to upload them to the remote repository. Conversely, git pull retrieves and integrates the latest remote changes into the local repository. This workflow is important for maintaining updated project copies and supporting collaboration.

3.1) Write a Windows Batch Script to automate the synchronization of files and folders between a source and destination directory using the XCOPY command and schedule it using Windows Task Scheduler.

Procedure:

Step 1: Open Command Prompt.

Step 2: Create two directories:

- *mkdir source*
- *mkdir destination*

Step 3: Navigate to the *source* directory and create sample files and folders.

Step 4: Return to the Desktop and create a batch file named replica.bat.

Step 5: Open the batch file using Notepad.

Step 6: Write the following script in the batch file:

```
@echo off
xcopy Source_Path Destination_Path /I /E /Y
echo Done.
```

Step 7: Save the batch file.

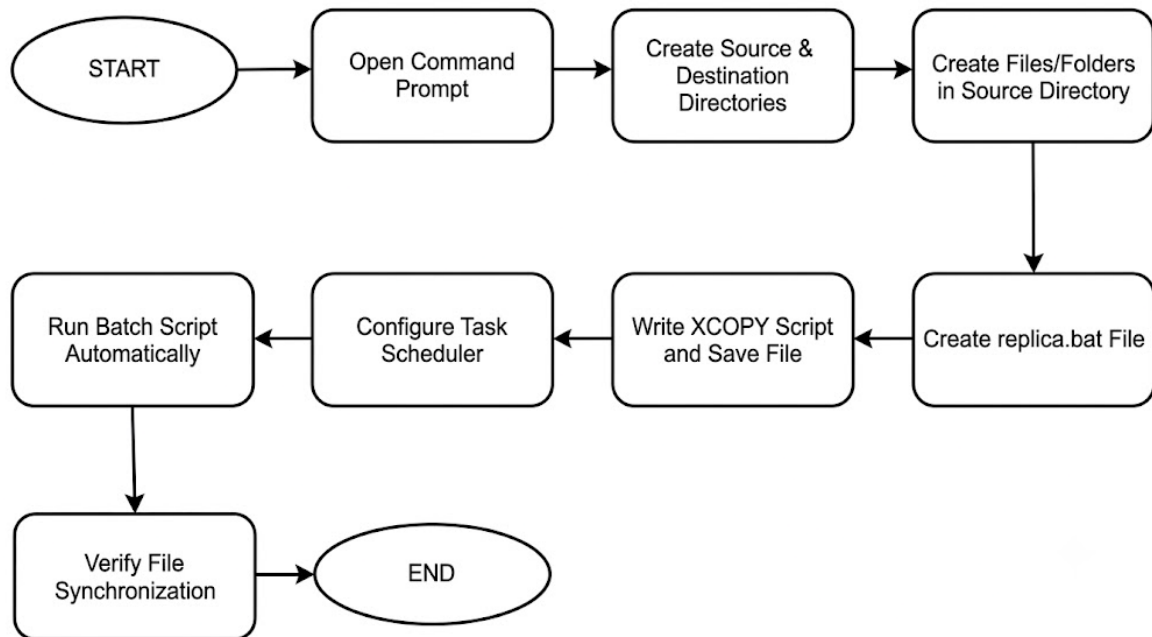
Step 8: Open Windows Task Scheduler and create a Basic Task.

Step 9: Configure the task to execute replica.bat automatically at a specified interval (e.g., every 5 minutes).

Step 10: Verify that synchronization occurs automatically.

Step 11: End the practical.

Block Diagram:



Source Code

```
echo @off
```

```
xcopy "D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works\OS\source" "D:\Prasoon Raj  
Shakya\BSc. CSIT\4th sem\Lab works\OS\destination"
```

```
echo done
```

Screenshots

Creating directory source and destination

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

Loading personal and system profiles took 8007ms.
(base) PS D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works\OS> mkdir source

Directory: D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works\OS

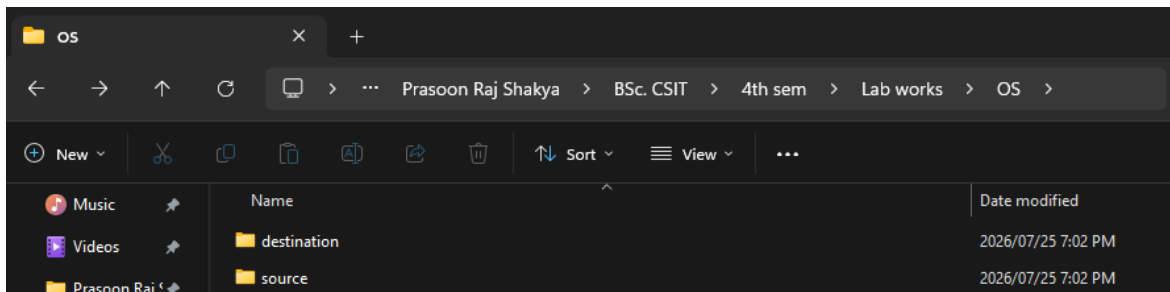
Mode                LastWriteTime         Length Name
----                -
d-----          2026/07/25   7:02 PM             source

(base) PS D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works\OS> mkdir destination

Directory: D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works\OS

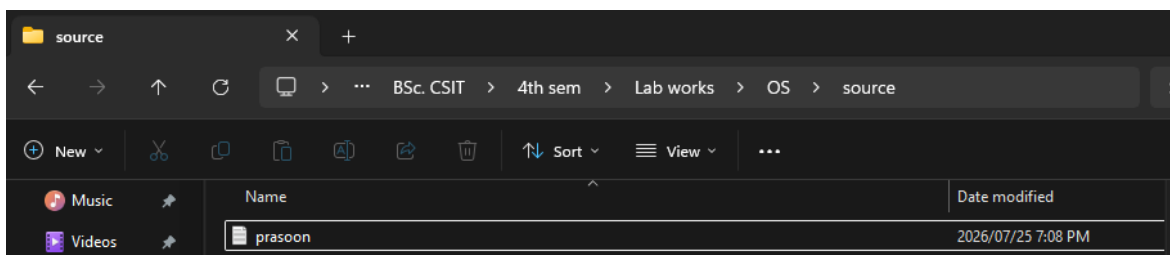
Mode                LastWriteTime         Length Name
----                -
d-----          2026/07/25   7:02 PM             destination

(base) PS D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works\OS>
```



Creating file prasoon.txt in source

```
Windows PowerShell
(base) PS D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works\OS\source> echo "">prasoon.txt
(base) PS D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works\OS\source>
```



replica.bat in Notepad

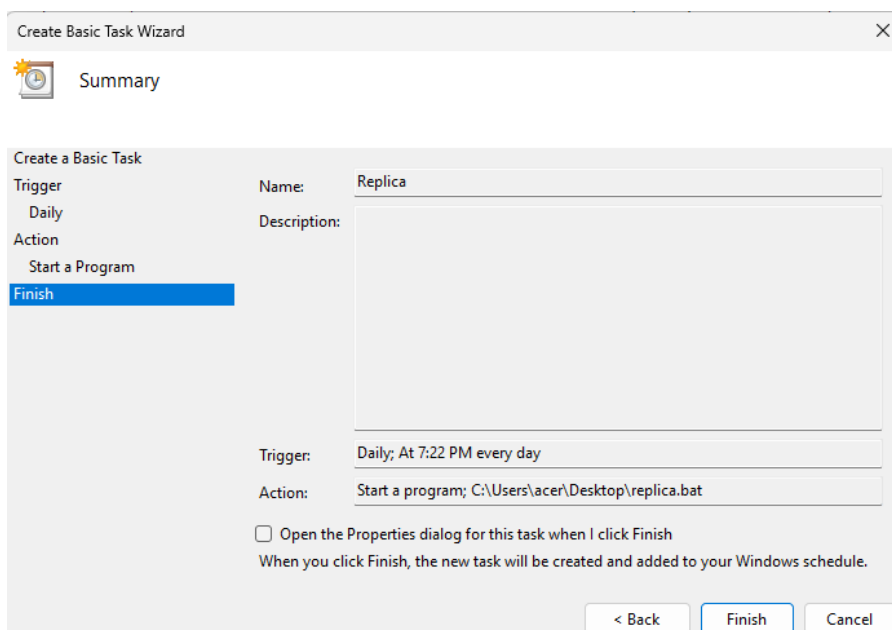
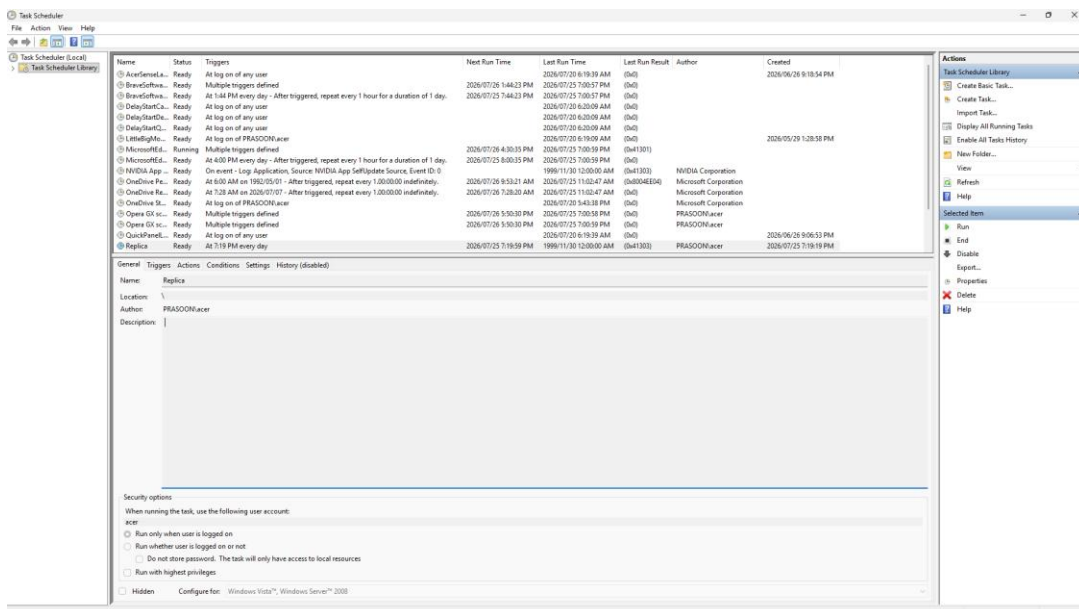
```
File Edit View

echo @off

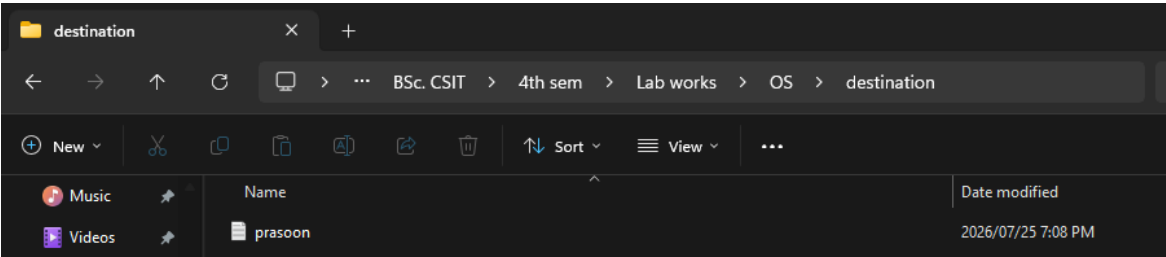
xcopy "D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works\OS\source"
"D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works\OS\destination"

echo done
```

Scheduling Using task scheduler



After the task scheduler executes replica.bat



3.2) Demonstrate remote repository synchronization using Git by performing push and pull operations between a local repository and a remote repository.

Procedure:

Step 1: Open the local Git repository (agent repo) in Command Prompt.

Step 2: Make changes by creating or modifying files in the local repository.

Step 3: Stage the changes using:

git add .

Step 4: Commit the changes using:

git commit -m "Commit message"

Step 5: Push the committed changes to the remote repository using:

git push

Step 6: Verify that the changes are uploaded to the remote repository.

Step 7: Make changes in the remote repository (or from another local copy).

Step 8: Update the local repository by executing:

git pull

Step 9: Verify that the latest changes are synchronized successfully.

Step 10: End the practical.

Screenshots

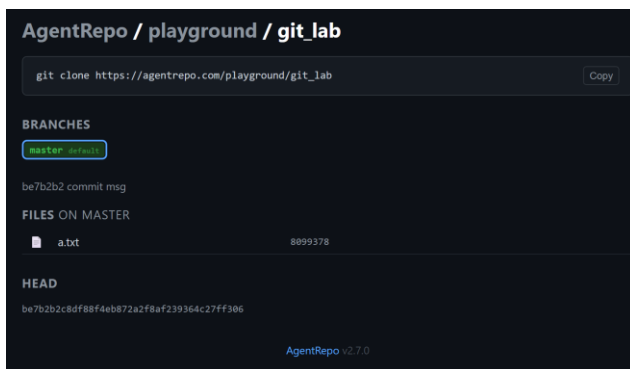
Git push

```
Windows PowerShell
(base) PS D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works\OS> mkdir git

Directory: D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works\OS

Mode                LastWriteTime         Length Name
----                -
d-----          2026/07/25   7:30 PM             git

(base) PS D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works\OS> cd git
(base) PS D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works\OS\git> echo "hello world">a.txt
(base) PS D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works\OS\git> git init
Initialized empty Git repository in D:/Prasoon Raj Shakya/BSc. CSIT/4th sem/Lab works/OS/git/.git/
(base) PS D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works\OS\git> git add .
(base) PS D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works\OS\git> git commit -m "commit msg"
[master (root-commit) be7b2b2] commit msg
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 a.txt
(base) PS D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works\OS\git> git remote add origin https://agentrepo.com/playground/git_lab
(base) PS D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works\OS\git> git push origin master
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 236 bytes | 236.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Unpacking objects: 100% (3/3), done.
To https://agentrepo.com/playground/git_lab
* [new branch]      master -> master
(base) PS D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works\OS\git> |
```



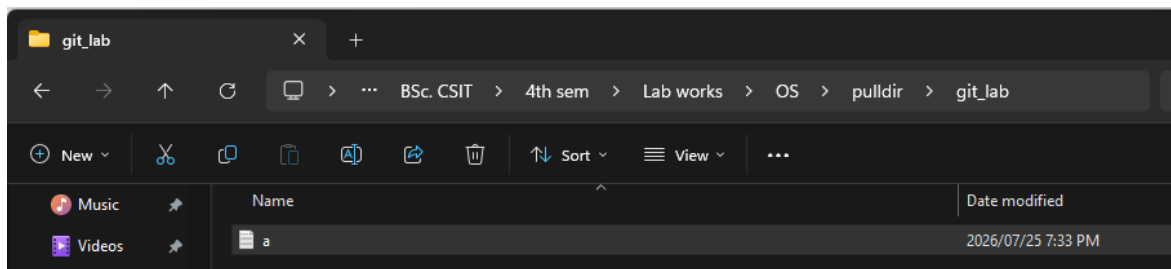
Git clone

```
Windows PowerShell
(base) PS D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works\OS\git> cd ..
(base) PS D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works\OS> mkdir pulldir

Directory: D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works\OS

Mode                LastWriteTime         Length Name
----                -
d-----          2026/07/25   7:32 PM             pulldir

(base) PS D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works\OS> cd pulldir
(base) PS D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works\OS\pulldir> git clone https://agentrepo.com/playground/git_lab
Cloning into 'git_lab'...
Unpacking objects: 100% (3/3), 216 bytes | 19.00 KiB/s, done.
(base) PS D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works\OS\pulldir> |
```



Git pull

```
Windows PowerShell
(base) PS D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works\OS\pulldir> git pull origin main
From https://github.com/ignotus-8/4th
 * branch          main          -> FETCH_HEAD
Already up to date.
(base) PS D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works\OS\pulldir> |
```

Conclusion

This experiment demonstrated automated synchronization of local files and folders using a Windows batch script, XCOPY command, and Windows Task Scheduler. It showed that scheduled execution can maintain a regularly updated copy of a source directory at a destination location. The experiment also demonstrated Git-based synchronization between local and remote repositories through push and pull operations. These activities provided practical understanding of automation, backup replication, version control, and remote repository management. Therefore, the objectives of the experiment were successfully achieved.

Lab 4: Exploring WSL—Ubuntu CLI Fundamentals and Command-Line Arguments in C

Theory

Windows Subsystem for Linux (WSL) allows a Linux distribution such as Ubuntu to run within the Windows operating system. It provides a Linux command-line environment for creating directories, managing files, editing programs, compiling source code, and executing applications. The basic WSL commands used in this lab are:

1. `pwd` – displays the current working directory.
2. `cd` – changes the current directory.
3. `mkdir` – creates a new directory.
4. `ls` – lists files and folders in a directory.
5. `nano` – opens a text editor to create or edit files.
6. `gcc` – compiles a C source file into an executable program.
7. `./test` – executes the compiled program named `test`.
8. `touch` – creates an empty file.
9. `cp` – copies files or directories.
10. `clear` – clears the terminal screen.

Write a C program to demonstrate the use of command-line arguments by adding two numbers passed through the command line.

In C programming, command-line arguments allow users to pass data to a program when it is executed from the terminal. These arguments are received through the `main()` function using the following syntax:

```
int main(int argc, char *argv[])
```

- `argc` (argument count) stores the total number of command-line arguments passed to the program, including the program name.

- `argv` (argument vector) is an array of strings that stores the command-line arguments. In this program, `argv[1]` and `argv[2]` contain the two numbers entered by the user.

Additionally,

- `atoi()` is a library function that converts a string into an integer. It converts the values in `argv[1]` and `argv[2]` so that they can be added mathematically.

Write a C program to demonstrate the use of command-line arguments in Ubuntu WSL by creating, editing, compiling, and executing a C program using the Linux command-line interface.

Procedure:

Step 1: Open Ubuntu in Windows Subsystem for Linux (WSL).

Step 2: Display the current working directory using: `pwd`

Step 3: Navigate to the Home directory (if required): `cd ~`

Step 4: Create a new directory: `mkdir test`

Step 5: Move into the newly created directory: `cd test`

Step 6: Create a C source file: `nano helloworld.c`

Step 7: Write the C program that accepts and displays command-line arguments, then save the file.

Step 8: Compile the program using the GCC compiler: `gcc helloworld.c -o helloworld`

Step 9: Execute the compiled program with command-line arguments: `./helloworld`

Step 10: Observe the output displaying the total number of arguments and their values.

Step 11: End the program.

Source Code:

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
int main(int argc, char *argv[]){
```

```
    int x=atoi(argv[1]);
```

```
    int y=atoi(argv[2]);
```

```
    printf("Sum = %d", x+y);
```

```
    printf("\n");
```

```
    return 0;
```

```
}
```

Screenshots

Creating a folder and .c file:

```
praseon@Praseon: ~/test
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

Loading personal and system profiles took 5826ms.
(base) PS C:\WINDOWS\system32> wsl
praseon@Praseon:/mnt/c/WINDOWS/system32$ cd ~
praseon@Praseon:~$ pwd
/home/praseon
praseon@Praseon:~$ mkdir test
praseon@Praseon:~$ ls
test
praseon@Praseon:~$ cd test
praseon@Praseon:~/test$ nano helloworld.c
praseon@Praseon:~/test$ ls
helloworld.c
```

Writing program to add 2 numbers passed from CLI:

```
GNU nano 8.7.1 helloworld.c
#include<stdio.h>
#include<stdlib.h>

int main(int argc, char *argv[]){
int x=atoi(argv[1]);
int y=atoi(argv[2]);
printf("Sum = %d",x+y);
printf("\n")
return 0
}
```

Compiling and running the file:

```
praseon@Praseon: ~/test
praseon@Praseon:~/test$ ls
helloworld helloworld.c
praseon@Praseon:~/test$ gcc helloworld.c -o helloworld
praseon@Praseon:~/test$ ./helloworld 11 8
Sum = 19
praseon@Praseon:~/test$ |
```

Conclusion

This experiment explored the Ubuntu command-line environment through WSL and demonstrated the creation, compilation, and execution of a C program. It provided practical experience with essential Linux commands for directory navigation and file management. The program successfully accepted numbers as command-line arguments and performed their addition after converting them into integer values. The experiment improved understanding of GCC compilation, program execution, and command-line input handling in C. Hence, the objectives of the experiment were successfully achieved.